

安全漏洞审计报告 · 第4轮

用户信息管理平台 业务逻辑漏洞审计报告

User Management Platform — Business Logic Vulnerability Assessment

目标系统：用户信息管理平台 (Flask + SQLite)

报告日期：2026-07-10

漏洞等级：严重 × 2 | 高危 × 2 | 中危 × 3

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 漏洞详情 — 7 项业务逻辑漏洞
3. 攻击演示
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 全部四轮汇总

1. 报告概述

1.1 本轮背景

前三轮修复了密码安全、SQL 注入、文件上传漏洞后，系统新增了 个人中心（**/profile**）和充值（**/recharge**）功能。但充值功能存在多处业务逻辑漏洞——代码在语法上没有错误，但业务规则出现严重缺陷，导致攻击者可以恶意篡改余额。

1.2 什么是业务逻辑漏洞？

业务逻辑漏洞 $\neq$ 代码语法错误。代码能正常运行，但业务规则设计不合理，导致用户可以做"系统不该允许"的操作，例如：给自己充任意多的钱、把余额改成负数、不登录就改别人余额。

2. 漏洞详情

2

● 严重

2

● 高危

3

● 中危

7/7

✓ 已修复

VULN-12：充值无金额上限

● 严重

位置：/recharge — POST 路由

描述：amount 参数没有上限限制，攻击者可一次充值 999999999999 让余额变成天文数字，完全破坏系统经济平衡。

攻击场景：普通用户给自己充 999999999999 元 → 余额变成 1 后面 13 个 0 → 账户余额失控。

VULN-13：充值可导致余额负数

● 高危

位置：/recharge — POST 路由

描述：amount 可为任意负数，充值值等于扣钱。如果输入比余额还大的负数，余额变成负数，代表用户以"欠债"，这在用户管理系统中不应存在。

攻击场景：充 -99999999999999 → 余额变成巨大的负数 → 系统负债。

VULN-14：未登录也可充值（越权操作）

● 高危

位置：/recharge — POST 路由

描述：任何人无需登录即可直接 POST 请求 /recharge 修改任意用户的余额，只需知道 user_id。攻击者可任意篡改所有用户的余额。

攻击场景：攻击者直接 `curl -X POST ... -d "user_id=admin&amount=0"` 无需任何身份认证。

VULN-15：注册错误信息泄露

● 中危

位置：/register — POST 路由

描述：注册重复用户名时，系统直接返回 SQLite 底层错误：UNIQUE constraint failed: users.username。攻击者可以用此接口探测哪些用户名已注册。

攻击场景：尝试注册 "admin" → 报错 UNIQUE → 确认 admin 已存在 → 枚举所有用户名。

VULN-16：搜索接口未登录泄露用户数据

● 中危

位置：/search — GET 路由

描述：无需登录即可搜索所有用户的邮箱和手机号。输入一个简单字母即可返回大量用户信息，造成隐私泄露。

攻击场景：/search?keyword=a → 返回所有用户名中包含 a 的用户的完整资料。

VULN-17：越权访问个人中心

● 严重

位置：/profile — GET 路由

描述：profile 接口从 URL 参数 user_id 获取用户 ID，攻击者只需修改 URL 中的数字即可查看任意用户的邮箱、手机、余额等敏感信息，无需任何权限校验。

攻击场景：/profile?user_id=2 → 直接查看 alice 的全部个人资料。

VULN-18：充值操作无反馈提示

● 中危

位置：/recharge — POST 路由

描述：充值无论是成功还是失败，都只默默重定向回个人中心页面，没有任何成功或失败提示。用户无法判断操作是否成功，也不知道失败原因。

攻击场景：充值超过上限 → 跳转回个人中心 → 余额不变 → 用户不知道为什么。

3. 攻击演示

3.1 充值天文数字

📌 攻击请求

```
curl -X POST http://localhost:5000/recharge \  
-d "user_id=1&amount=999999999999"
```

👁️ 攻击结果

余额从 0.0 变为 10000000000699.0 (一千万亿)

3.2 充负值制造负数余额

📌 攻击请求

```
curl -X POST http://localhost:5000/recharge \  
-d "user_id=1&amount=-999999999999"
```

👁️ 攻击结果

余额从正数变为 -89999999999300.0 (负数)

3.3 未登录越权充值

📌 攻击请求

```
# 无需任何 Cookie/Session  
curl -X POST http://localhost:5000/recharge \  
-d "user_id=2&amount=99999"
```

👁️ 攻击结果

alice 的余额被未登录攻击者修改为 12221212.0

3.4 注册探测用户名

📌 攻击请求

```
curl -X POST http://localhost:5000/register \  
-d "username=admin&password=x&email=x&phone=x"
```

👁️ 攻击结果

返回: UNIQUE constraint failed: users.username
攻击者确认 "admin" 已注册

3.5 越权访问他人资料

攻击请求

```
curl http://localhost:5000/profile?user_id=2
```

攻击结果

无需登录即可看到 `alice` 的邮箱、手机、余额

3.6 充值无提示

攻击请求

```
curl -X POST http://localhost:5000/recharge \  
-d "user_id=1&amount=99999"
```

攻击结果

只跳转回 `/profile` , 没有任何成功或失败提示信息

4. 修复方案

4.1 VULN-12 : 充值金额上限

```
if abs(amount_num) > 10000:
    return redirect(f"/profile?user_id={user_id}") # 单笔不超过 ±10000
```

4.2 VULN-13 : 防止余额为负数

```
new_balance = current_balance + amount_num
if new_balance < 0:
    return redirect(f"/profile?user_id={user_id}") # 余额不低于 0
```

4.3 VULN-14 : 充值需要登录

```
if "username" not in session:
    return redirect("/login") # 未登录的充值请求全部拒绝
```

4.4 VULN-15 : 隐藏错误详情

```
# 修复前：暴露 SQLite 底层错误
return render_template("register.html", error=f"注册失败: {e}")

# 修复后：通用提示
return render_template("register.html", error="注册失败，用户名可能已被占用")
```

4.5 VULN-17 : 修复越权访问 (**profile**从**session**获取用户)

```
# 修复前：从 URL 参数获取 user_id (可越权)
user_id = request.args.get("user_id")
c.execute("SELECT ... WHERE id = ?", (user_id,))

# 修复后：从 session 获取当前用户 (不可越权)
username = session.get("username")
c.execute("SELECT ... WHERE username = ?", (username,))
```

4.6 VULN-18 : 充值反馈提示

```
# 修复前：静默跳转，无任何提示
return redirect(f"/profile?user_id={user_id}")

# 修复后：URL参数携带中文提示信息 (需URL编码)
return redirect("/profile?msg=" + quote("充值成功，当前余额: 500.0"))
return redirect("/profile?msg=" + quote("充值失败，单次金额不能超过10000"))
```

5. 修复前后对照

/recharge 路由完整对比

✖ 修复前

```
def recharge():  
    # 无登录检查  
    user_id = request.form.get("user_id")  
    amount = request.form.get("amount", "0")  
    UPDATE users SET balance = balance + ?  
    # 无上限、无负数检查
```

✔ 修复后

```
def recharge():  
    if "username" not in session: redirect  
    user_id = request.form.get("user_id")  
    amount = request.form.get("amount", "0")  
    if abs(amount) > 10000: redirect  
    if new_balance < 0: redirect  
    UPDATE users SET balance = ?  
    # 三层防护：登录 + 上限 + 余额非负
```


6. 修复验证

#	测试	结果
1	充值 99999 (超过 10000 上限) → 被拒绝	✓ 通过
2	充负值导致余额负数 → 被拒绝	✓ 通过
3	未登录发起充值 → 跳转登录页	✓ 通过
4	重复注册 → 通用错误提示 , 不暴露 UNIQUE 信息	✓ 通过
5	正常充值 5000 → 成功到账	✓ 通过
6	正常扣款 -3000 且余额仍 ≥ 0 → 成功	✓ 通过

7. 全部四轮汇总

轮次	审计内容	漏洞数	严重	高危	中危	状态
第 1 轮	密码安全	8	4	-	4	✓
第 2 轮	SQL 注入	2	2	-	-	✓
第 3 轮	文件上传	1	1	-	-	✓
第 4 轮	业务逻辑	7	2	2	3	✓
合计		18	9	2	7	全部已修复

本轮总结

业务逻辑漏洞是最容易被忽视的安全问题。代码没有语法错误，SQL 也没有注入风险，但缺少业务规则校验导致用户可以做不合理的事。修复的核心是 5 条规则：

- 💰 金额控制：单次充值上限 ± 10000 ，余额不能为负
- 🔑 操作鉴权：修改余额必须登录
- 🎯 信息最小化：错误提示不暴露数据库内部信息
- 🛡️ 对象级权限：个人中心从 session 获取用户，不接受 URL 参数
- 🗣️ 操作反馈：所有操作必须返回明确的成功/失败提示

安全漏洞审计报告 · 第4轮 — 业务逻辑安全 | 报告日期：2026-07-10

项目地址：<https://github.com/sencly-book/user-management>